

03 Cloud Setup & Initial Deployment

Data Engineering Project: Serverless Spotify ETL Pipeline - Cloud Setup & Initial Deployment

This section details the foundational steps for deploying our Serverless Spotify ETL (Extract, Transform, Load) pipeline using **AWS cloud services**. We'll cover setting up an AWS account, configuring budgeting, structuring S3 for data storage, and deploying the initial extraction and transformation logic using AWS Lambda.

1. AWS Account Setup and Cost Management

To begin, an **AWS account is essential**. While other cloud providers can be used, we will focus on AWS.

- **Creating an AWS Account:**
 - Search for "AWS" on Google and navigate to their cloud computing platform.
 - You will be prompted for credit or debit card information during sign-up, but **AWS offers a 12-month free trial**, so you will not be charged initially.
 - The account creation process is straightforward, requiring basic information.
- **Creating a Budgeting Alarm (Best Practice):**
 - A budgeting alarm is crucial to monitor costs and prevent unexpected charges, even though the services chosen generally fall under the free trial.
 - **Ensure your AWS region is set to North Virginia.** This region is often the first to receive new AWS updates and features.
 - **Steps to set up a billing alarm:**
 1. Click on your name in the AWS console and select "Billing Dashboard".
 2. Go to "Billing Preferences" and **tick the "Receive billing alerts" option**.
 3. Search for "CloudWatch" in the AWS console.
 4. Navigate to "Alarms" and then "Billing".
 5. Click "Create alarm".
 6. Select "Metrics" > "Billing" > "Total Estimated Charges" > "USD".
 7. Set your desired **threshold (e.g., 5 USD)**. If estimated charges exceed this, you'll receive a notification.
 8. Click "Next," then "Create new topic".
 9. Provide a topic name (e.g., "AWS billing alarm") and **your email address** to receive notifications.
 10. Click "Create topic," then "Next," and finally "Create alarm".
 - This alarm acts as a safety measure, notifying you if your costs exceed the set limit.

2. AWS S3 Bucket Configuration for Data Storage

Amazon S3 (Simple Storage Service) is a highly **scalable object storage** service capable of storing virtually unlimited files. We will use S3 to store both our raw JSON data and the transformed CSV files.

- **Creating an S3 Bucket:**
 - Search for "S3" in the AWS console.

- Click "Create bucket".
- **Bucket Name:** Choose a globally **unique name** across the entire AWS universe (e.g., `spotify-etl-project-yourname`).
- **Region:** Select **North Virginia** (us-east-1) to align with other services and leverage the latest features.
- Keep other settings as default and click "Create bucket".
- **Structuring Folders within the S3 Bucket:**
 - To organize data effectively, create distinct folders inside your bucket.
 - **Raw Data Folder:** `raw_data`
 - Purpose: Store the raw JSON files exactly as extracted from the Spotify API.
 - Sub-folders within `raw_data` :
 - `to_processed` : For raw data files that are awaiting processing.
 - `processed` : For raw data files that have already been processed and transformed.
 - **Rationale:** This structure ensures that only new, unprocessed data is handled, preventing redundant processing of the same files. Data is moved from `to_processed` to `processed` after successful transformation.
 - **Transformed Data Folder:** `transformed_data`
 - Purpose: Store the transformed and structured data.
 - Sub-folders within `transformed_data` :
 - `album_data` : For album-specific transformed data.
 - `songs_data` : For song-specific transformed data.
 - `artist_data` : For artist-specific transformed data.
 - This hierarchical structure (`bucket > raw_data/transformed_data > sub-folders`) provides a clear organization for the ETL pipeline.

3. Deploying the Data Extraction Lambda Function

AWS Lambda is a **compute service** that allows you to run small amounts of code or perform small transformation jobs triggered by events. It's ideal for our serverless ETL architecture.

- **Creating the Lambda Function:**
 - Search for "Lambda" in the AWS console and open it in a new tab.
 - Click "Create function".
 - **Function Name:** Provide a descriptive name (e.g., `Spotify-API-Data-Extract`).
 - **Runtime:** Select **Python 3.8**.
 - **Permissions:** Choose "Create a new role with basic Lambda permissions".
 - Click "Create function".
- **Securing Credentials with Environment Variables:**
 - **Best Practice:** Never hardcode sensitive information like Client IDs and Secret IDs directly into your code.
 - Instead, use **environment variables** within Lambda.
 - **Steps:**
 1. In your Lambda function's UI, go to "Configuration" > "Environment variables".
 2. Click "Edit" and "Add environment variable".
 3. Add `CLIENT_ID` as a key and your Spotify Client ID as its value.

4. Add `CLIENT_SECRET` as a key and your Spotify Secret ID as its value.
 5. Save the changes.
- **Accessing in Code:** Use the `os` module (`import os`) and `os.environ.get('YOUR_VARIABLE_NAME')` to retrieve these values securely in your Python code.
 - **Handling External Python Packages (Lambda Layers):**
 - AWS Lambda environments do not natively include all external Python libraries (e.g., `spotipy`, `pandas`).
 - **Solution:** Use **Lambda Layers** to provide these external dependencies.
 - **Steps for `spotipy`:**
 1. In the Lambda console, search for "Layers".
 2. Click "Create layer".
 3. Provide a layer name (e.g., `Spotify_Layer`).
 4. **Upload the `spotipy` zip file** (which contains the installed `spotipy` package).
 5. Select **Compatible Architectures:** `x86_64` and **Runtimes:** `Python 3.8`.
 6. Click "Create".
 7. Go back to your Lambda function, scroll down, and click "Add a layer".
 8. Choose "Custom layers" and select your `Spotify_Layer`.
 9. Click "Add". This makes `spotipy` available to your function.
 - **Writing the Extraction Code:**
 - The code will:
 - Import `os`, `spotipy`, `spotipy.oauth2.SpotifyClientCredentials`, `json`, `datetime`.
 - Initialize `spotipy.Spotify` using Client ID and Secret ID retrieved from environment variables.
 - Define the Spotify "Top Global Songs" playlist URL.
 - Extract the playlist URI by string splitting the URL.
 - Call `sp.playlist_tracks(playlist_uri)` to fetch raw JSON data.
 - **Loading Raw Data to S3:**
 - Use the `boto3` library (AWS SDK for Python) to interact programmatically with AWS services like S3.
 - Initialize an S3 client: `s3_client = boto3.client('s3')`.
 - Use `s3_client.put_object()` to store the raw JSON data.
 - **Parameters:**
 - **Bucket :** Your S3 bucket name.
 - **Key :** The desired path within the bucket (e.g., `raw_data/to_processed/`).
 - **Body :** The raw JSON data converted to a string using `json.dumps()`.
 - **Dynamic Filenames:** Generate unique filenames using the current date and time (e.g., `spotify_raw_YYYY-MM-DD-HH-MM-SS.json`) to avoid overwriting and maintain historical records.
 - Import `datetime` from `datetime`.
 - Use `datetime.now().strftime()` to format the timestamp.
 - **Granting S3 Permissions (IAM Role):**

- **Problem:** Initially, the Lambda function might throw an `AccessDenied` error when trying to write to S3.
- **Reason:** The default Lambda execution role typically only has basic permissions and lacks explicit S3 write access.
- **Solution:** Modify the Lambda's IAM role.
 1. In Lambda, go to "Configuration" > "Permissions".
 2. Click on the role name (e.g., `Spotify-API-Data-Extract-role- ...`). This opens the IAM console.
 3. Click "Add permissions" > "Attach policies".
 4. Search for `S3` and attach the `AmazonS3FullAccess` policy for now (in a production environment, you would use a more granular policy).
 5. Click "Attach policy". This grants the Lambda function permission to interact with S3.
- **Adjusting Lambda Timeout:**
 - **Problem:** Lambda functions have a default timeout (e.g., 3 seconds), which can cause execution failures for longer tasks.
 - **Solution:** Increase the timeout.
 1. In Lambda, go to "Configuration" > "General configuration".
 2. Click "Edit" and increase the timeout (e.g., to 1 minute 30 seconds).
 3. Click "Save".
- **Testing and Deployment:**
 - After every code change, click "Deploy" to save your changes.
 - Click "Test" and configure a simple test event (e.g., `{}`) to execute the function.
 - Observe the logs for errors and successful execution.
 - Upon successful execution, raw JSON files will appear in your `raw_data/to_processed` S3 folder.

```
import json
import os
import spotipy
from spotipy.oauth2 import SpotifyClientCredentials
import boto3
from datetime import datetime

def lambda_handler(event, context):

    client_id = os.environ.get('client_id')
    client_secret = os.environ.get('client_secret')

    client_credentials_manager = SpotifyClientCredentials(client_id=client_id,
client_secret=client_secret)
    sp = spotipy.Spotify(client_credentials_manager = client_credentials_manager)
    playlists = sp.user_playlists('spotify')

    playlist_link = "https://open.spotify.com/playlist/37i9dQZEVXbNG2KDcFcKOF?
si=1333723a6eff4b7f"
    playlist_URI = playlist_link.split("/")[-1].split("?")[0]
```

```

spotify_data = sp.playlist_tracks(playlist_URI)

cilent = boto3.client('s3')

filename = "spotify_raw_" + str(datetime.now()) + ".json"

cilent.put_object(
    Bucket="spotify-etl-project-darshil",
    Key="raw_data/to_processed/" + filename,
    Body=json.dumps(spotify_data)
)

```

4. Deploying the Data Transformation Lambda Function

This second Lambda function will **process the raw JSON files** from `raw_data/to_processed`, **transform them**, and **store them as structured CSVs** in the `transformed_data` folders.

- **Creating the Transformation Lambda Function:**
 - Create a new Lambda function.
 - **Function Name:** `Spotify-Transformation-Load-Function`.
 - **Runtime: Python 3.8.**
 - **Role:** Select the **existing IAM role** that was used for the extraction function, as it already has the necessary S3 permissions (`Spotify-API-Data-Extract-role- ...`).
 - Click "Create function".
- **Processing Strategy:**
 - The goal is to **process new data only**.
 - The function will:
 1. List all files in the `raw_data/to_processed` folder.
 2. Read each JSON file.
 3. Apply transformation logic.
 4. Store transformed data in `transformed_data`.
 5. **Move the processed raw file** from `raw_data/to_processed` to `raw_data/processed`.
- **Core Transformation Logic (Python Code):**
 - **Import** `boto3, json, pandas as pd, io.StringIO, datetime`.
 - **Listing Files in S3:**
 - Initialize S3 client: `s3_client = boto3.client('s3')`.
 - Use `s3_client.list_objects_v2(Bucket=bucket_name, Prefix=f'{raw_data_path}/to_processed/')`.
 - Iterate through `response['Contents']` to get each `file['Key']` (which is the full S3 path of the file).
 - **Filter:** Ensure only `.json` files are processed by checking the file extension.
 - **Reading JSON from S3:**
 - For each file key, use `s3_client.get_object(Bucket=bucket_name, Key=file_key)`.
 - Read the `Body` of the response and parse it using `json.loads()`.

- Store the loaded JSON data and the file key in lists (e.g., `spotify_data_list`, `spotify_keys_list`).
- **Modularizing Transformations into Functions:**
 - Encapsulate the transformation logic for albums, artists, and songs into separate Python functions (e.g., `extract_album_data(data)`, `extract_artist_data(data)`, `extract_song_data(data)`).
 - These functions should take the raw JSON data as input and return a list of dictionaries for each entity (album, artist, song).
- **Applying Transformations and Creating DataFrames:**
 - Loop through `spotify_data_list`.
 - For each raw data entry, call the album, artist, and song extraction functions.
 - **Convert the resulting lists of dictionaries into Pandas DataFrames** using `pd.DataFrame.from_dict()`.
- **Pandas Layer:**
 - **Problem:** Lambda does not natively support the `pandas` library.
 - **Solution:** Similar to `spotipy`, add a Lambda Layer for Pandas.
 - Go to Lambda Layers, and add the **"AWS SDK Pandas" layer**. This is an AWS-provided layer.
 - Attach it to your transformation Lambda function.
- **Data Cleaning and Type Conversion:**
 - **Dates:** Convert date columns (e.g., `release_date`, `added_at`) to `datetime` objects using `pd.to_datetime()` for proper temporal operations.
 - **Duplicates:** Remove duplicate records from `album_df` and `artist_df` based on their respective IDs using `df.drop_duplicates(subset=['id'], inplace=True)` to maintain data integrity.
- **Loading Transformed Data to S3 (CSV):**
 - For each DataFrame (album, artist, song):
 - Generate a unique filename for the CSV using `datetime` (e.g., `album_transformed_YYYY-MM-DD-HH-MM-SS.csv`).
 - Define the target S3 key (path) within `transformed_data` (e.g., `transformed_data/album_data/`).
 - Use `io.StringIO` to create an in-memory text buffer.
 - Write the DataFrame to this buffer in CSV format using `df.to_csv(buffer, index=False)`.
 - Use `s3_client.put_object(Bucket=bucket_name, Key=target_key, Body=buffer.getvalue())` to upload the CSV string to S3.
- **Iterative Debugging:**
 - The development process often involves encountering and resolving errors (e.g., `No module name`, `AccessDenied`, `Timeout`).
 - **Key Debugging Steps:**
 - Always **deploy** changes after coding.
 - Use the "Test" feature in Lambda to run your function.
 - Check the CloudWatch logs for detailed error messages.
 - Adjust IAM permissions and timeouts as necessary.

- **Outcome:** After successful execution, transformed CSV files will be stored in `transformed_data/album_data`, `transformed_data/artist_data`, and `transformed_data/songs_data` within your S3 bucket. The raw files from `to_processed` will then be moved to `processed` to indicate they've been handled.

```
import json
import boto3
from datetime import datetime
from io import StringIO
import pandas as pd

def album(data):
    album_list = []
    for row in data['items']:
        album_id = row['track']['album']['id']
        album_name = row['track']['album']['name']
        album_release_date = row['track']['album']['release_date']
        album_total_tracks = row['track']['album']['total_tracks']
        album_url = row['track']['album']['external_urls']['spotify']
        album_element = {
            'album_id': album_id, 'name': album_name, 'release_date': album_release_date,
            'total_tracks': album_total_tracks, 'url': album_url
        }
        album_list.append(album_element)
    return album_list

def artist(data):
    artist_list = []
    for row in data['items']:
        for key, value in row.items():
            if key == "track":
                for artist in value['artists']:
                    artist_dict = {'artist_id': artist['id'],
                        'artist_name': artist['name'], 'external_url': artist['href']}
                    artist_list.append(artist_dict)
    return artist_list

def songs(data):
    song_list = []
    for row in data['items']:
        song_id = row['track']['id']
        song_name = row['track']['name']
        song_duration = row['track']['duration_ms']
        song_url = row['track']['external_urls']['spotify']
        song_popularity = row['track']['popularity']
        song_added = row['added_at']
        album_id = row['track']['album']['id']
        artist_id = row['track']['album']['artists'][0]['id']
        song_element = {
            'song_id': song_id, 'song_name': song_name, 'duration_ms': song_duration, 'url': song_ur
l,
```

```

        'popularity': song_popularity, 'song_added': song_added, 'album_id': album_id,
        'artist_id': artist_id
    }
    song_list.append(song_element)

    return song_list

def lambda_handler(event, context):
    s3 = boto3.client('s3')
    Bucket = "spotify-etl-project-darshil"
    Key = "raw_data/to_processed/"

    spotify_data = []
    spotify_keys = []
    for file in s3.list_objects(Bucket=Bucket, Prefix=Key)['Contents']:
        file_key = file['Key']
        if file_key.split('.')[-1] == "json":
            response = s3.get_object(Bucket = Bucket, Key = file_key)
            content = response['Body']
            jsonObject = json.loads(content.read())
            spotify_data.append(jsonObject)
            spotify_keys.append(file_key)

    for data in spotify_data:
        album_list = album(data)
        artist_list = artist(data)
        song_list = songs(data)

        album_df = pd.DataFrame.from_dict(album_list)
        album_df = album_df.drop_duplicates(subset=['album_id'])

        artist_df = pd.DataFrame.from_dict(artist_list)
        artist_df = artist_df.drop_duplicates(subset=['artist_id'])

        #Song Dataframe
        song_df = pd.DataFrame.from_dict(song_list)

        album_df['release_date'] = pd.to_datetime(album_df['release_date'])
        song_df['song_added'] = pd.to_datetime(song_df['song_added'])

        songs_key = "transformed_data/songs_data/songs_transformed_" +
str(datetime.now()) + ".csv"
        song_buffer=StringIO()
        song_df.to_csv(song_buffer, index=False)
        song_content = song_buffer.getvalue()
        s3.put_object(Bucket=Bucket, Key=songs_key, Body=song_content)

        album_key = "transformed_data/album_data/album_transformed_" +
str(datetime.now()) + ".csv"
        album_buffer=StringIO()
        album_df.to_csv(album_buffer, index=False)
        album_content = album_buffer.getvalue()

```



```
s3.put_object(Bucket=Bucket, Key=album_key, Body=album_content)

artist_key = "transformed_data/artist_data/artist_transformed_" +
str(datetime.now()) + ".csv"
artist_buffer=StringIO()
artist_df.to_csv(artist_buffer, index=False)
artist_content = artist_buffer.getvalue()
s3.put_object(Bucket=Bucket, Key=artist_key, Body=artist_content)

s3_resource = boto3.resource('s3')
for key in spotify_keys:
    copy_source = {
        'Bucket': Bucket,
        'Key': key
    }
    s3_resource.meta.client.copy(copy_source, Bucket, 'raw_data/processed/' +
key.split("/")[-1])
    s3_resource.Object(Bucket, key).delete()
```
